

Stackademic · [Follow publication](#)

★ Member-only story

How I Built a Custom AI Document Assistant That Understands 1000s of PDFs and Talks Like a Human

5 min read · Jul 3, 2025



Abdul Ahad

Follow



Listen



Share



More

Forget basic search. I designed a Retrieval-Augmented Generation (RAG) system that reads technical documents, extracts diagrams, interprets text, and answers questions like a domain expert — all using open-source AI.

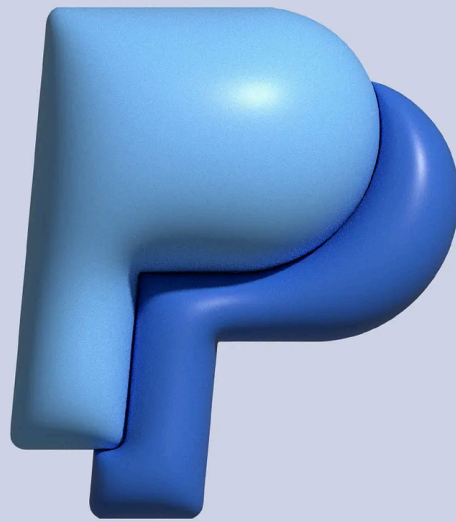


Photo by [Farhat Altaf](#) on [Unsplash](#)

The Real Problem: Why Traditional Search Fails for Technical Docs

Ever tried finding a specific piece of info from 500+ PDFs?

Even with filenames like `Report_Final_v2_NEW_Latest.pdf`, good luck.

Search tools don't **understand** content. They match keywords. That's not enough for:

- Extracting insights from scientific papers
- Summarizing legal contracts
- Comparing product specs

That's when I decided to build something smarter — a **multimodal RAG AI** that reads, understands, and answers questions from thousands of PDFs.

Step 1: Structuring the Project Directory

Organizing everything properly up front helped me scale this from 10 docs to 10,000.

```
ai-doc-assistant/
├── ingest/
│   ├── extract_text.py
│   └── extract_images.py
├── process/
│   ├── chunk_text.py
│   └── embed_chunks.py
├── index/
│   └── vector_store.py
├── backend/
│   ├── qa_chain.py
│   └── server.py
├── interface/
│   └── ui.py
```

Each module had one responsibility.

Ingest → Process → Index → Answer → Display.

Step 2: Extracting Text From PDFs Using PyMuPDF

Text is the gold mine. So I extracted it page by page, preserving metadata.

```
import fitz

def extract_text(file_path):
    doc = fitz.open(file_path)
    pages = []
    for i, page in enumerate(doc):
        text = page.get_text()
        pages.append({
            "file": file_path,
            "page": i + 1,
            "text": text
        })
    return pages
```

This gave me full control: filenames, page references, and selective inclusion.

Step 3: Extracting and Storing Diagrams From PDFs

Visuals carry meaning — especially in research and product manuals.

So I extracted all embedded images.

```
def extract_images(pdf_path, output_dir):
    doc = fitz.open(pdf_path)
    for page_index in range(len(doc)):
        images = doc[page_index].get_images(full=True)
        for img_index, img in enumerate(images):
            xref = images[img_index][0]
            base_image = doc.extract_image(xref)
            image_bytes = base_image["image"]
            image_filename = f"{output_dir}/{page_index}_{img_index}.png"
            with open(image_filename, "wb") as img_file:
                img_file.write(image_bytes)
```

Later, these diagrams were embedded using CLIP and stored alongside text.

Step 4: Chunking Text With Overlap for RAG-Friendly Context

Large LLMs can't take full documents — so we chunk them.

```
def chunk_text(text, size=500, overlap=100):
    chunks = []
    for i in range(0, len(text), size - overlap):
        chunk = text[i:i + size]
        chunks.append(chunk)
    return chunks
```

Overlap ensures that context doesn't break between chunks.
It's essential for coherent answers.

Step 5: Generating Embeddings for Text Chunks

Now, I transformed the chunks into vectors using `sentence-transformers`.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

def embed_chunks(chunks):
    return model.encode(chunks)
```

These vectors represent **meaning**, not just keywords.
So later, we can retrieve the most relevant concepts, even if phrased differently.

8 Python Libraries That Turned Me Into a Backend Ninja

From Sleepless Debugging Nights to Seamless Automation — These Tools Changed Everything

python.plainenglish.io

Step 6: Building a Fast Vector Search Engine With FAISS

I used FAISS to store and search embeddings efficiently.

```
import faiss
import numpy as np

def build_index(embeddings):
    dim = embeddings.shape[1]
    index = faiss.IndexFlatL2(dim)
    index.add(embeddings)
    return index
```

Once built, this allowed me to run 10,000+ doc searches in under a second.

Step 7: Adding CLIP Embeddings for Image-Based Search

To make the assistant multimodal, I indexed diagrams using CLIP.

```
from transformers import CLIPProcessor, CLIPModel
from PIL import Image

clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

def embed_image(image_path):
    image = Image.open(image_path)
    inputs = processor(images=image, return_tensors="pt")
    with torch.no_grad():
```

```
image_features = clip_model.get_image_features(**inputs)
return image_features.squeeze().numpy()
```

Now the bot could answer queries like “show me the process diagram for system reboot”.

Step 8: Building the Retrieval + Answering Chain With LangChain

Once the top documents were retrieved, I fed them into an LLM.

```
from langchain.chains import RetrievalQA
from langchain.vectorstores import FAISS
from langchain.llms import Ollama

retriever = FAISS.load_local("index", embeddings=model)
llm = Ollama(model="mistral")

qa = RetrievalQA.from_chain_type(llm=llm, retriever=retriever)
```

Query it:

```
qa.run("What's the warranty coverage of Product X?")
```

Boom — human-like answers with cited documents.

Step 9: Citing Sources in Answers

To increase trust, I appended sources to every answer.

```
def format_response(answer, docs):
    refs = "\n".join([f"{doc.metadata['file']}] (p{doc.metadata['page']})" for doc in docs])
    return f"{answer}\n\nSources:\n{refs}"
```

This made it feel more like a lawyer than a chatbot.

Every claim was traceable.

Step 10: Creating a Local API for Integration

I wrapped the whole pipeline in FastAPI:

```
from fastapi import FastAPI
from pydantic import BaseModel

class Query(BaseModel):
    question: str

app = FastAPI()

@app.post("/ask")
def ask(query: Query):
    response = qa.run(query.question)
    return {"answer": response}
```

I could now connect this backend to web apps, Slack, or even voice.

Step 11: Building a Web UI With Gradio

To demo it to clients and teammates, I created a slick UI:

```
import gradio as gr

def answer_question(q):
    return qa.run(q)

gr.Interface(fn=answer_question, inputs="text", outputs="text", title="AI PDF Assistant").launch()
```

They asked real questions from company manuals — and the bot *nailed it*.

Step 12: Handling File Uploads and Live Ingestion

Users could drop new PDFs on the UI, and they were instantly processed and indexed.

```
@app.post("/upload")
async def upload_pdf(file: UploadFile):
    save_path = f"./docs/{file.filename}"
    with open(save_path, "wb") as f:
        f.write(await file.read())
    # Extract, chunk, embed, and update FAISS index
```

This made the assistant **self-updating**.

New docs = new knowledge.

13 Python Features That Feel Like Magic Once You Start Using Them

These are the kind of tools that make you whisper, “Wait... Python can do that?”

python.plainenglish.io

Step 13: Deploying It Locally With Ollama + Docker

For full control, I containerized everything.

Dockerfile :

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["uvicorn", "backend.server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Now it runs on any laptop, without the cloud.

Step 14: Adding Long-Term Memory With SQLite

I gave the assistant memory:

```
import sqlite3

def save_interaction(question, answer):
    conn = sqlite3.connect("history.db")
    conn.execute("INSERT INTO log (q, a) VALUES (?, ?)", (question, answer))
    conn.commit()
```

Later, I used this to retrain the model and improve responses.

Step 15: Final Thoughts — From Search to Reasoning

This wasn't just search. This was **reasoning** over documents, charts, and audio.
It could:

- Compare product specs
- Summarize sections of a manual
- Find visuals from a paragraph
- Support users with personalized answers

I didn't just build a bot. I built an **AI teammate**.

“Want a pack of prompts that work for you and save hours? click [here](#)”

Do subscribe to my publication on [Gh0st](#) for more content...

Level Up Your Python Skills Instantly!  Download the Ultimate Python Cheat Sheet — 100% FREE & Beginner-Friendly!

MUST READ TO GET MORE THRILL ... 

I Tried 7 Viral AI Tool Sites, Here's What Actually Worked I've been programming in Python for over four years, and if there's one thing I've learned, it's this: tools are only... ai.plainenglish.io	
The Python Mistake I Made That No Tutorial Warned Me About And how it almost wrecked my automation mindset before it even began python.plainenglish.io	
This AI Site Said It Could Design My Brand in 60 Seconds. So I Tried It. You know that feeling when you sit down to finally “start your brand,” and 30 minutes later, you’re still stuck... medium.com	
CODRIFT : A New Publication on Medium Looking for your contribution Where automation, AI, and real-world coding collide. medium.com	

[medium.com](#)

If this story brought you any value, a clap (or two!) would mean the world to me!

Thank you for reading and for helping me grow this community!

Thank you for being a part of the community

Before you go:

- Be sure to **clap** and **follow** the writer 🍌
- Follow us: [X](#) | [LinkedIn](#) | [YouTube](#) | [Newsletter](#) | [Podcast](#) | [Twitch](#)
- [Start your own free AI-powered blog on Differ](#) 🚀
- [Join our content creators community on Discord](#) 👤
- For more content, visit [plainenglish.io](#) + [stackademic.com](#)

AI

Technology

Coding

Programming

Data Science



Follow

Published in Stackademic

52K followers · Last published 4 hours ago

Stackademic is a learning hub for programmers, devs, coders, and engineers. Our goal is to democratize free coding education for the world.



Follow

Written by Abdul Ahad

2.9K followers · 37 following

Web developer | Coder | Freelancer | Entrepreneur | Teacher | Software Engineer Owner of Codrift.

Responses (28)



Erdogan T

What are your thoughts?



Dan Dragos Stefanescu
Jul 10



Thanks for this nice article. Would you mind putting all on GitHub and sharing the link?



44



1 reply

[Reply](#)



Yuvarajkumaryadavk
Jul 5



Why did you choose sqlite?



28



1 reply

[Reply](#)



Ashishgoyalji
Jul 10



Excellent

 19 [Reply](#)

[See all responses](#)

More from the list: "LLMd"

Curated by Erdogan T

 Ramakrushna Mohapatra
Top 10 LLM & RAG Projects for Your AI...
★ · Aug 3

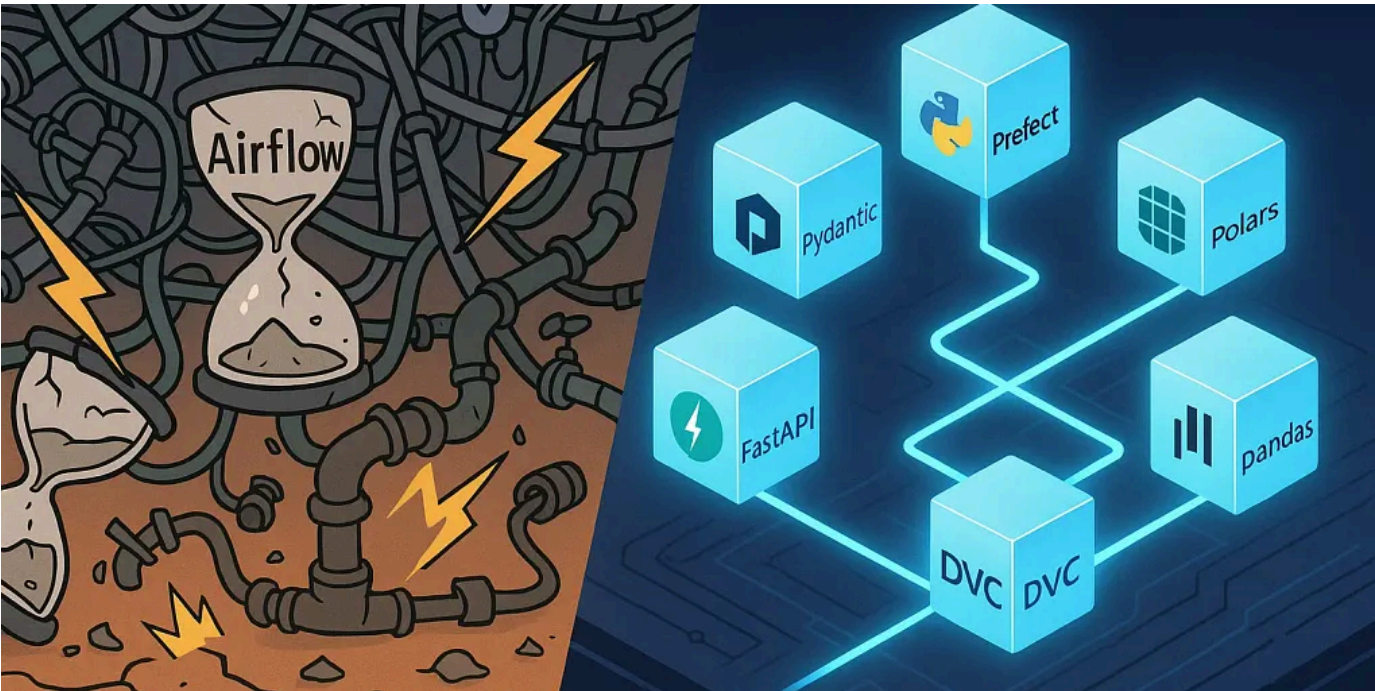
 In Coding N... by Civil Lear...
Google's New LLM Runs on Just 0.5 GB RAM—...
★ · Aug 15

 In Level Up C... by Fareed ...
Building the Entire RAG Ecosystem and...
★ · Aug 6

 Sau Sheong 
How to fine-tune an LLM
★ · Jun 18

[View list](#)

More from Abdul Ahad and Stackademic



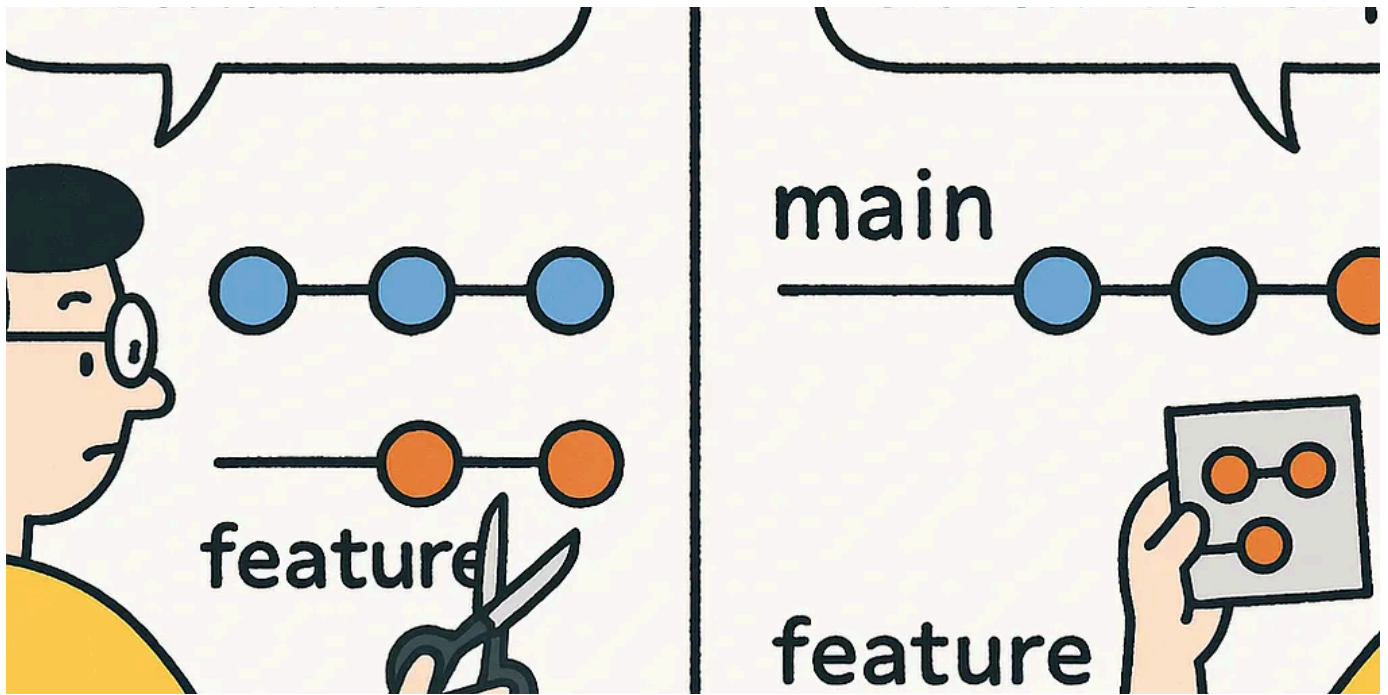
 In Python in Plain English by Abdul Ahad

I Replaced Airflow with These 6 Python Tools—My DAGs Are Now 10x Faster

Airflow was great—until it wasn't. These leaner Python tools brought back joy to my pipelines and shaved hours off my runtime.

★ Aug 7  325



 In Stackademic by Subodh Shetty

Git Rebase - Explained Like You're New to Git

When you work in Git, you'll often make a branch to build a feature or fix a bug. While you're working, other people keep adding new...

★ Aug 11 👍 767 💬 15

🔖 ...



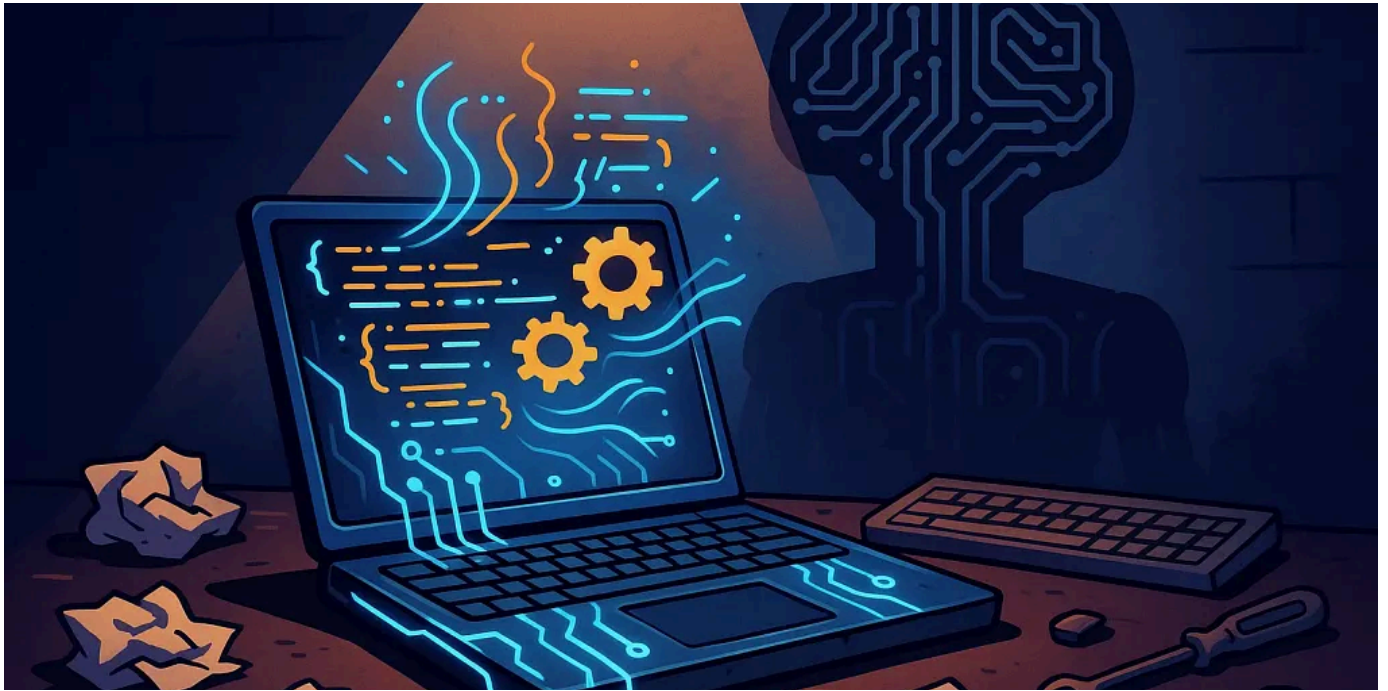
 In Stackademic by Mark Henry

My 13 Sources of Income as a Software Engineer

Because One Paycheck Is Too Fragile to Rely On

★ Aug 18 👍 1.2K 💬 28

🔖 ...



 In Stackademic by Abdul Ahad

How I Turned My Laptop into a Full-Time AI Developer—And Watched It Outperform Me

If someone had told me last year that I'd be shipping products faster than entire startup teams—without hiring a soul—I wouldn't have...

★ Aug 1 🖱️ 373 💬 5

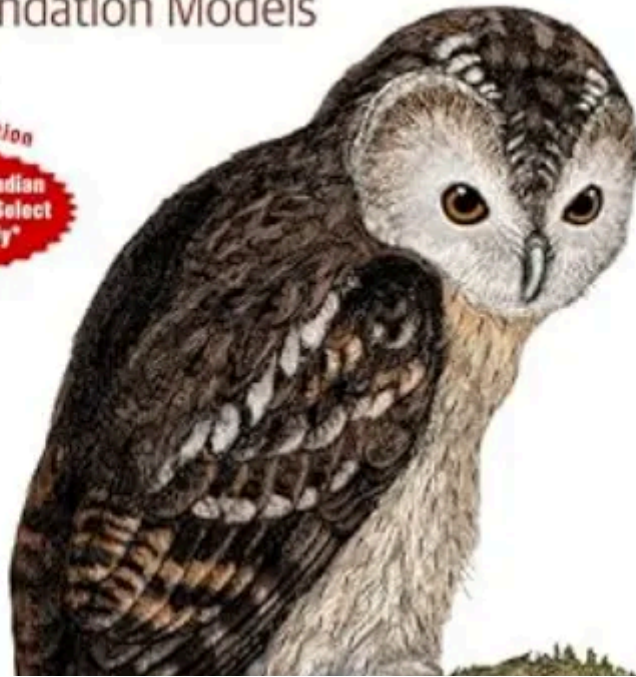
[See all from Abdul Ahad](#)

[See all from Stackademic](#)

Recommended from Medium

with Foundation Models

Full Colour Edition
For Sale in the Indian
Subcontinent & Select
Countries Only*

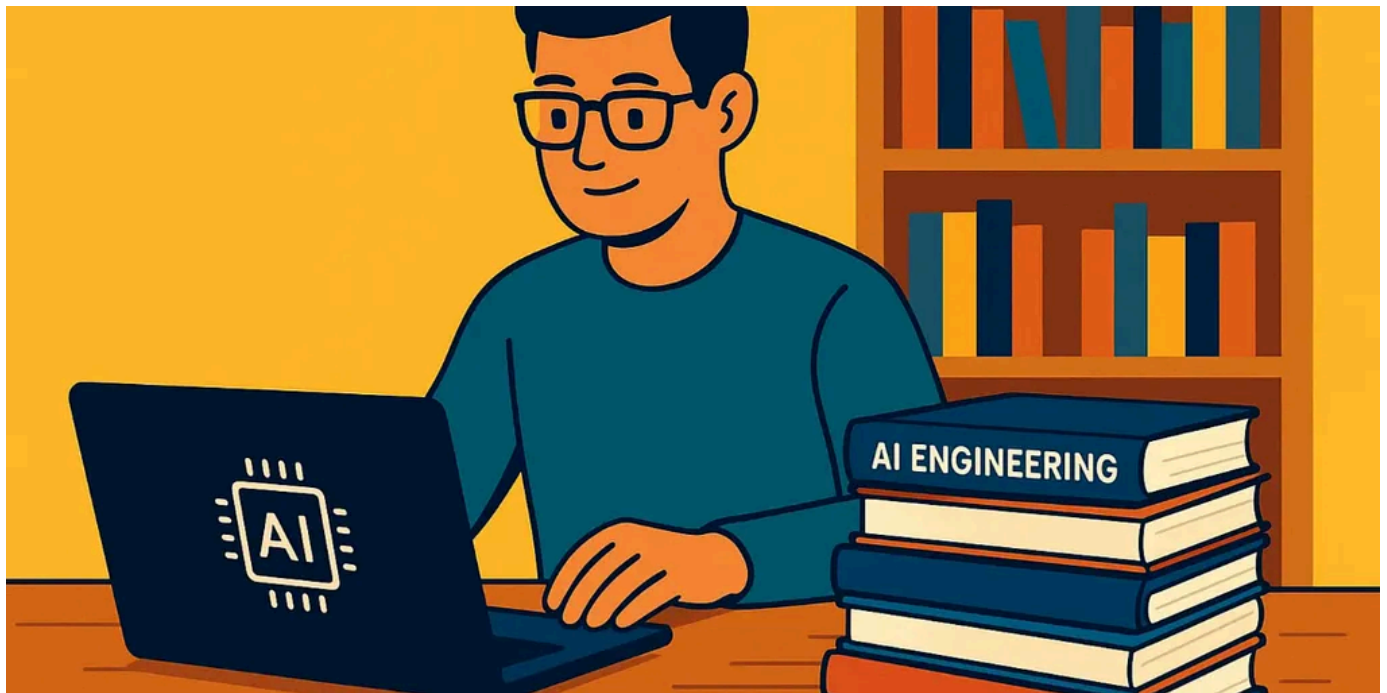
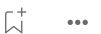


 In Android Alchemy by Prakash Sharma

Stop watching random AI courses: Complete Guide to AI Engineering

Ultimate guide for AI Engineering preparation , Six months preparation guide —

★ Jul 30 🖱 243 💬 14

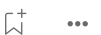


 Joe Njenga

9 Books Every AI Engineer Should Read (To Go Fully Professional)

Picking an AI engineering book and reading it from start to finish is tough!

★ Jul 23 🖱 404 💬 9



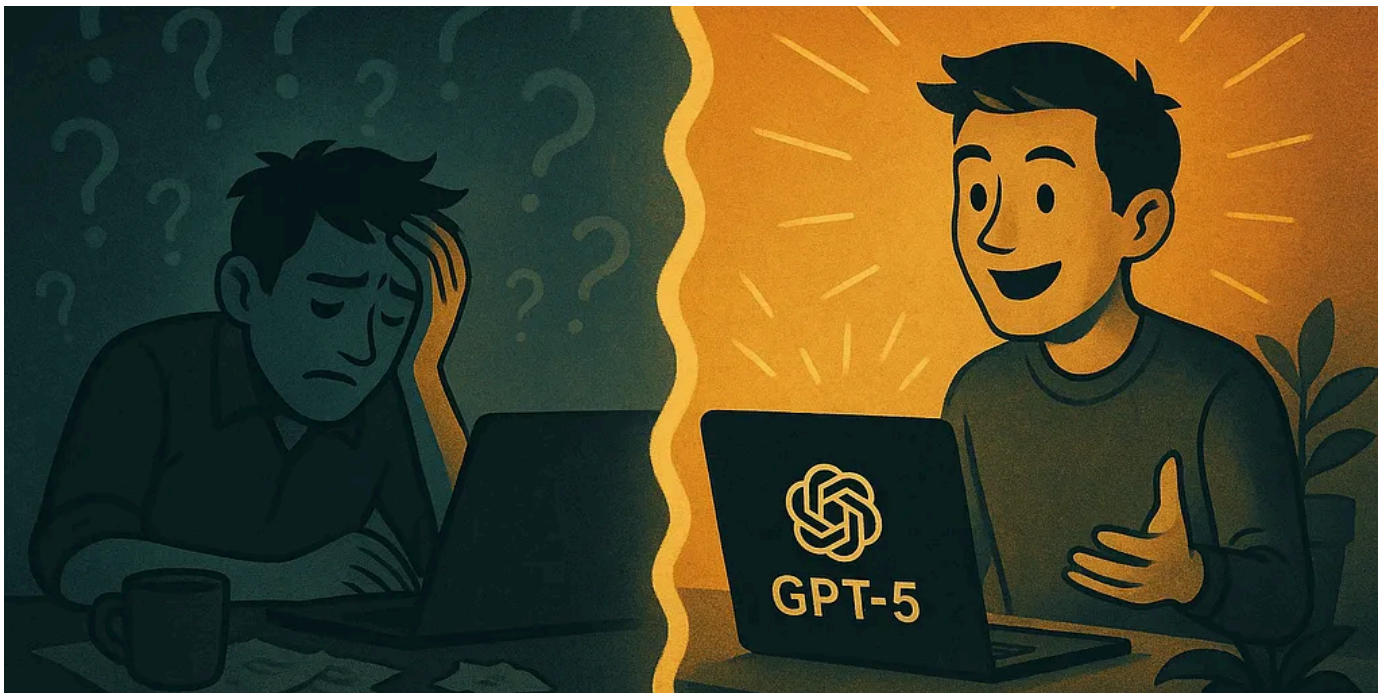
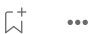


py In Python in Plain English by Mayuresh K

Architecture of AI-Driven Systems: What Every Technical Architect Should Know

A Step-by-Step Guide to Integrating External AI Services into Commercial Systems with Fallback Strategies

Aug 6 221 7



AI In Artificial Corner by The PyCoach

You're using GPT-5 wrong! Here's how to be ahead of 99% of users

Here's how to make the most out of GPT-5

Aug 20 1.2K 27



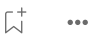


In Vibe Coding by Alex Dunlop

How I Turn Claude Into a Systems Engineering Genius With One Prompt

How a legendary systems engineer's lecture became my most powerful AI prompt

★ Aug 23 🖱 489 💬 8

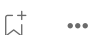


0 Orami

I Tested Every AI Code Generator—Here's the Winner

After 6 months and 500+ hours of rigorous testing, the results may surprise you

★ Aug 8 🖱 1.8K 💬 71



See more recommendations